

CS509 Laboratory Repository

Repository Overview

This repository contains **Assignment 03 - Buddy Task**. It follows the same menu-driven setup used for the individual Assignment 03 project, but implements two different algorithms:

- Gradient Descent
- Maxflow-Mincut using Dinic's algorithm

The assignment specifies generic one-variable polynomial Gradient Descent and directed-capacity Maxflow-Mincut tests. filecite turn0file0 L1-L10 filecite turn0file0 L16-L22

Student / Buddy Details

- **Assignment Mode:** Buddy / Double
- **Student 1:** Rajan Jha
- **Entry Number:** 2026AIM1019
- **Student 2:** Deepak kumar
- **Entry Number:** 2026CSM1012

Language and Environment

- **Language:** C++17
- **Compiler:** GNU g++
- **Compiler flags:** `-std=c++17 -O2 -Wall -Wextra -pedantic`
- **Build system:** GNU Make
- **Operating system:** Ubuntu Linux
- **External libraries:** None

Directory Structure

```
assignment_03_buddy/  
├── src/  
│   ├── common.h  
│   ├── graph_io.cpp  
│   ├── gradient_descent.cpp  
│   ├── gradient_descent.h  
│   ├── maxflow.cpp  
│   └── maxflow.h
```

```
├── tests/
│   ├── gradient_descent/
│   │   ├── gd_01.txt
│   │   ├── gd_02.txt
│   │   ├── gd_03.txt
│   │   ├── gd_04.txt
│   │   └── gd_05.txt
│   └── maxflow/
│       ├── maxflow_01.txt
│       ├── maxflow_02.txt
│       ├── maxflow_03.txt
│       ├── maxflow_04.txt
│       └── maxflow_05.txt
├── outputs/
│   ├── gradient_descent/
│   └── maxflow/
├── driver/
│   └── main.cpp
├── Makefile
└── README.md
```

Common Wrapper: Build and Usage

Compile:

```
make
```

Run:

```
./assignment_03_buddy
```

Menu:

```
1. Gradient Descent
2. Maxflow-Mincut
0. Exit
```

The driver remains in a loop, so another test can be selected without restarting the executable.

Assignment 03 - Buddy Task

Assignment Mode

Objective

Implement and test Gradient Descent and Maxflow-Mincut using the required input formats, test sizes and runtime measurement rules.

Part A - Gradient Descent

Gradient Descent uses:

$$f(x) = c_0 + c_1x + c_2x^2 + \dots + c_dx^d$$

$$f'(x) = c_1 + 2c_2x + \dots + d c_d x^{(d-1)}$$

$$x_{\text{new}} = x - \text{learning_rate} * f'(x)$$

The same implementation supports all required polynomial degrees rather than using separate hard-coded functions. filecite turn0file0 L2-L8

Required Tests

Test	Degree	Objective	Initial x	Learning Rate	Tolerance	Max Iterations	Expected x*
gd_01	2	$x^2 - 6x + 9$	0	0.10	1e-6	5,000	3
gd_02	4	$x^4 + 2x^2$	2	0.02	1e-6	10,000	0
gd_03	6	$0.1x^6 + 0.5x^4 + x^2$	2	0.02	1e-6	20,000	0
gd_04	8	$0.01x^8 + 0.05x^6 + 0.2x^4 + x^2$	2	0.01	1e-8	50,000	0
gd_05	10	$0.002x^{10} + 0.01x^8 + 0.05x^6 + 0.2x^4 + x^2$	2	0.005	1e-10	100,000	0

These are the required degree and parameter values. filecite turn0file0 L59-L67 filecite turn0file0 L68-L107

Input Format

```
DEGREE d
COEFFICIENTS c0 c1 c2 ... cd
INITIAL_X x0
LEARNING_RATE alpha
```

```
TOLERANCE epsilon
MAX_ITERATIONS n
```

The coefficients are stored from the constant term through the highest power. filecite turn0file0 L116-L129

Results

File	Degree	x0	Rate	Tol.	Max Iter.	Exp. x*	Exp. f(x*)	Actual x	Actual f(x)	Iter. / Time	Status
gd01.txt	2	0	0.1	1e-06	5,000	3	0	2.9999995063	0.0000000000	70 / 0.000380 ms	Fail
gd02.txt	4	2	0.02	1e-06	10,000	0	0	0.0000002379	0.0000000000	180 / 0.001202 ms	Pass
gd03.txt	6	2	0.02	1e-06	20,000	0	0	0.0000004835	0.0000000000	349 / 0.002854 ms	Pass
gd04.txt	8	2	0.01	1e-08	50,000	0	0	0.0000000050	0.0000000000	948 / 0.009334 ms	Pass
gd05.txt	10	2	0.005	1e-10	100,000	0	0	0.0000000000	0.0000000000	2364 / 0.026730 ms	Pass

Part B - Maxflow-Mincut

Dinic's algorithm is used. It builds a level graph using BFS and sends blocking flow through the residual graph. The final residual graph is used to find the vertices reachable from the source, which gives one valid minimum cut.

The assignment recommends Dinic because of the larger graph sizes. filecite turn0file0 L17-L22

Required Tests

Test File	V	E	Graph Type
maxflow_01.txt	10	17	Directed capacity graph
maxflow_02.txt	100	197	Directed capacity graph
maxflow_03.txt	1,000	1997	Directed sparse capacity graph

Test File	V	E	Graph Type
maxflow_04.txt	10,000	19997	Directed sparse capacity graph
maxflow_05.txt	50,000	99997	Directed sparse capacity graph

The required sizes are 10, 100, 1,000, 10,000 and 50,000 vertices. filecite turn0file0 L45-L57

Input Format

```

V E
u0 degree neighbor1 capacity1 neighbor2 capacity2 ...
...
u(V-1) degree ...
SOURCE s
SINK t

```

Only original outgoing capacity edges are stored. Reverse residual edges are created by the algorithm.
 filecite turn0file0 L146-L161

Results

File	V	E	Source	Sink	Exp. Flow	Actual Flow	Cut Capacity	Time	Status
maxflow_01.txt	10	17	0	9	13	13	13	0.008333 ms	Pass
maxflow_02.txt	100	197	0	99	13	13	13	0.028422 ms	Pass
maxflow_03.txt	1,000	1,997	0	999	13	13	13	0.182662 ms	Pass
maxflow_04.txt	10,000	19,997	0	9999	13	13	13	1.927363 ms	Pass
maxflow_05.txt	50,000	99,997	0	49999	13	13	13	9.376648 ms	Pass

Runtime Measurement

For both algorithms, input reading and setup happen before the timer.

Only the selected algorithm is timed.

For Gradient Descent, the timed section contains the iterative optimization.

For Maxflow-Mincut, the timed section contains the Dinic maximum-flow computation and excludes file reading.

This follows the assignment requirement that setup work remain outside the measured algorithm time.
 filecite turn0file0 L113-L114

Complexity

Gradient Descent

For polynomial degree d and I iterations:

- Time: $O(d \times I)$
- Space: $O(d)$

Dinic Maxflow

The residual graph uses adjacency lists:

- Space: $O(V + E)$
- Running time depends on the graph structure; Dinic is used for the required larger graphs.

Expected Output

Gradient Descent reports:

```
Algorithm: Gradient Descent
Degree: <degree>
Final x: <value>
Final f(x): <value>
Iterations: <value>
Converged: true/false
Execution time: <value> ms
```

The required output fields are specified by the assignment. filecite turn0file0 L137-L145

Maxflow-Mincut reports:

```
Algorithm: Maxflow-Mincut
Source: <source>
Sink: <sink>
Maximum flow: <value>
Minimum cut capacity: <value>
Source side: ...
Sink side: ...
Cut edges:
u v capacity
...
Execution time: <value> ms
```

The maximum flow and minimum cut capacity must be equal. Different valid minimum-cut partitions are possible.

filecite turn0file0 L172-L186

References

1. Stanford CS231n – Loss Functions and Optimization. filecite turn0file0 L7-L10
2. MIT OpenCourseWare 6.046J – Max Flow, Min Cut. filecite turn0file0 L21-L22
3. CS509 Assignment 03 specification.